

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
КАРПАТСЬКИЙ НАЦІОНАЛЬНИЙ УНІВЕРСИТЕТ
ІМЕНІ ВАСИЛЯ СТЕФАНІКА
Кафедра комп'ютерних наук та інформаційних систем

Лабораторна робота № 8

з курсу

Крос-платформне
програмування Python

на тему:

«Декоратори та ітератори.»

Виконав:

студент групи

КН-31

Коновалов А.С.

«___» _____ 2026 р.

(підпис)

Перевірила:

ст. викладач кафедри КНІС

к.т.н. Семаньків М.В.

«___» _____ 2026 р.

(оцінка, підпис)

м. Івано-Франківськ

2026

Лабораторна робота № 8

Тема: Декоратори та ітератори.

Мета роботи: Навчитися створювати та застосовувати декоратори функцій, а також реалізовувати власні класи-ітератори для послідовної обробки даних.

Теоретичні відомості:

Декоратор у Python - це функція, яка приймає іншу функцію та повертає нову функцію-обгортку, додаючи додаткову поведінку без зміни вихідного коду. Застосування декоратора записують над функцією за допомогою синтаксису `@decorator`. Обгортка (wrapper) може виконувати код до і після виклику оригінальної функції, змінювати аргументи або результат. Декоратори широко використовуються для логування, перевірки умов, форматування виведення та повторного використання коду.

Ітератор - об'єкт, що реалізує методи `__iter__()` та `__next__()`. Метод `__next__()` повертає наступний елемент послідовності, а при завершенні генерує виняток `StopIteration`. Це дозволяє використовувати об'єкт у циклі `for`. Клас-ітератор зберігає поточний стан обходу (наприклад, поточне число та кінець діапазону) і видає елементи один за одним, що економить пам'ять порівняно зі створенням повного списку.

Хід роботи:

У ході виконання лабораторної роботи було створено два Python-сценарії: `lab8_task1.py` та `lab8_task2.py`.

Завдання 1 - Декоратори

У файлі `lab8_task1.py` реалізовано базові декоратори з методички (`my_decorator`, `dec_1`, `dec_2`, `dec_3`), а також практичні завдання: `makebold/makeitalic` для HTML-тегів, декоратор очищення тексту для перевірки паліндрома (фраза «Я несу гусеня»), декоратор перевірки унікальності значень словника перед обміном ключів і значень, декоратор нормалізації вхідних

даних для функції перевірки унікальності елементів. Продемонстровано, що порядок декораторів впливає на результат: `@makebold @makeitalic` дає `<b><i>текст</i></b>`, а зворотний порядок — `<i><b>текст</b></i>`.

Завдання 2 - Класи-ітератори

У файлі `lab8_task2.py` реалізовано клас `Iteration` з методички, який видає числа від 1 до `end` включно. Створено клас `RangeStepIterator` для проміжку з початком, кінцем і кроком. Наприклад, `RangeStepIterator(2, 10, 2)` дає `[2, 4, 6, 8, 10]`, а `RangeStepIterator(10, 0, -3)` — `[10, 7, 4, 1]`.

Висновки:

У ході виконання лабораторної роботи №8 опановано механізм декораторів як засобу розширення поведінки функцій без їхньої модифікації.

Реалізовано декоратори для форматування тексту, перевірки паліндромів, валідації словників та нормалізації вхідних даних.

Створено класи-ітератори для генерації числових послідовностей з заданим кроком.

У результаті роботи набуто досвіду застосування декораторів і ітераторів для побудови гнучких та ефективних Python-програм.